

Universidade Estadual do Tocantins (UNITINS)
Sistemas de Informação — Sistemas Distribuídos — 7º período, 2026/2

Processamento distribuído de grande volume de dados com arquitetura mestre/workers

Pré-projeto

Integrantes

Alêkson Castro
Italo Beckman
Matheus Nardi

Professor: Prof. Mailson Oliveira

Palmas – TO, 22/10/2026

Sumário

1 Problema e cenário escolhido **2**

2 Por que é um problema distribuído **2**

3 Arquitetura proposta **2**

4 Exploração dos conjuntos de dados **3**

5 Ambiente e tecnologias **4**

6 Métricas de sucesso e experimentos planejados **5**

7 Cronograma **6**

8 Divisão de papéis **6**

9 Riscos e desafios **6**

1 Problema e cenário escolhido

Entre os quatro cenários propostos pelo professor, o grupo escolheu o de **processamento distribuído de grande volume de dados com arquitetura mestre/workers**. O problema é processar um conjunto de dados grande demais para ser tratado por uma única máquina em tempo razoável, dividindo o trabalho entre várias máquinas que processam partes dos dados ao mesmo tempo e depois combinam seus resultados.

Objetivo. Construir um sistema mestre/workers que distribua o processamento de um grande volume de dados entre várias máquinas e avaliar experimentalmente seu desempenho (*speedup*), o balanceamento de carga entre máquinas heterogêneas e a tolerância a falhas de workers.

Este pré-projeto tem caráter **exploratório** em duas frentes: o **conjunto de dados** (Seção 4), ainda em análise, e o **ambiente de execução** (Seção 5), cujas opções foram avaliadas e que já está definido. A pergunta que o sistema vai responder e o tipo de processamento (agregação, contagem, filtragem...) são decisões posteriores, tomadas após a definição do dataset.

2 Por que é um problema distribuído

Uma única máquina tem limites de processamento (núcleos de CPU), de memória e de leitura em disco. Quando o volume de dados cresce, o tempo de processamento cresce junto e, a partir de certo ponto, os dados nem cabem na memória de uma máquina só. Distribuir o processamento contorna esses limites: cada máquina recebe uma parte dos dados, e o trabalho total é feito em paralelo.

A distribuição, porém, traz desafios que não existem em uma máquina só: máquinas com desempenho diferente, falhas durante a execução e o custo de mandar dados pela rede. Esses desafios são o foco dos experimentos planejados (Seção 6).

3 Arquitetura proposta

O sistema segue a arquitetura **mestre/trabalhador** (*master-worker*) [5], no modelo de paralelismo de dados popularizado pelo MapReduce [3] (Figura 1):

1. o **mestre** divide os dados em pedaços (*chunks*) e distribui as tarefas;
2. os **workers** processam seus pedaços em paralelo e devolvem resultados parciais pequenos (uma soma, uma contagem), nunca os dados brutos;
3. o mestre **agrega** os resultados parciais e produz a resposta final.

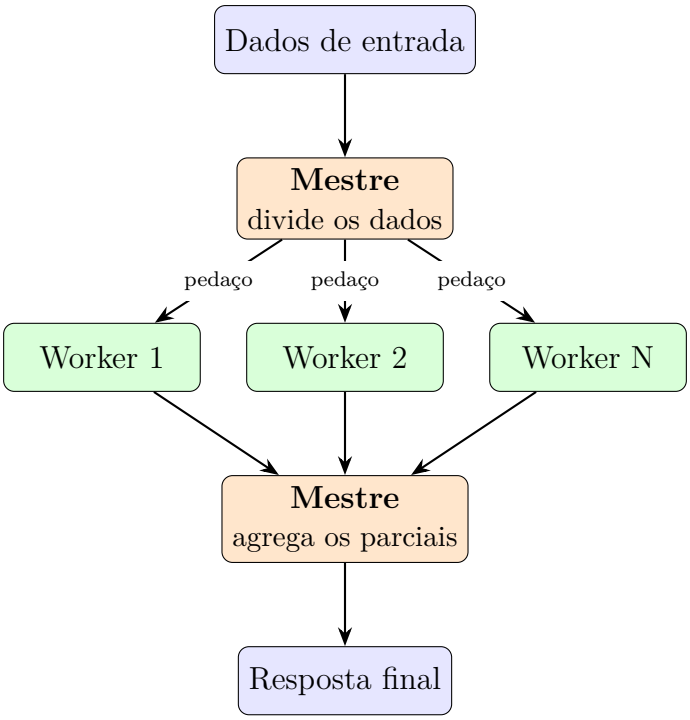


Figura 1: Fluxo da arquitetura mestre/workers.

Há duas formas de entregar os pedaços aos workers, e o grupo vai comparar as duas:

Estática: os dados são divididos em N partes iguais, uma por worker. É simples, mas se um worker for mais lento, todos esperam por ele.

Dinâmica (*self-scheduling*): os dados são divididos em muitos pedaços pequenos, e o mestre envia o próximo pedaço a cada worker que termina. Workers mais rápidos processam mais pedaços, e ninguém fica ocioso [1].

Diferentemente do cenário de balanceamento de carga, que distribui *requisições* contínuas com foco em tempo de resposta, este cenário distribui *pedaços de dados* de um trabalho com começo e fim (processamento em lote), com foco em terminar mais rápido.

4 Exploração dos conjuntos de dados

O dataset precisa, antes de tudo, **se encaixar no paralelismo de dados**. Os critérios de escolha são:

1. **Registros independentes:** cada linha pode ser processada sem olhar as outras, o que permite cortar os dados em pedaços de qualquer tamanho.
2. **Resultado parcial combinável:** soma, contagem, mínimo e máximo se combinam diretamente; a média também, se cada worker devolver soma e contagem. Já a *mediana* não pode ser obtida das medianas dos pedaços, um exemplo dos limites do modelo.
3. **Volume suficiente:** cerca de 100 MB para validar e de 1 a 5 GB para os experimentos; abaixo disso, o custo de distribuir supera o ganho.

4. **Acesso aberto e reprodutível:** download público, com o script de obtenção versionado no repositório.

Dataset	**Perguntas possíveis**	**Processamento**
Microdados do ENEM (INEP) [4]	Média de notas por estado, tipo de escola ou faixa de renda	Agregação por grupo
NYC Taxi Trip Records [6]	Corridas e receita por hora do dia ou por região	Agregação e filtragem
Dumps da Wikipédia	Palavras mais frequentes	Contagem (*word count*)
Dados sintéticos (*seed* fixa)	Qualquer, com volume controlado	Qualquer

Tabela 1: Datasets candidatos.

O grupo priorizou **primeiro a técnica e depois o negócio**: o núcleo do trabalho (distribuição, agregação e medições) não depende do tema dos dados, desde que o dataset atenda aos critérios acima. Por isso, o desenvolvimento pode começar com dados sintéticos enquanto os candidatos reais são analisados. O dataset ainda será decidido; a tendência é combinar dados reais, que dão sentido à pergunta, com sintéticos, que permitem variar o volume nos experimentos de escala. Os dados não serão versionados no Git, apenas os scripts que os obtêm ou geram.

5 Ambiente e tecnologias

Distribuição Linux. Existem distribuições próprias para clusters, como a **PelicanHPC** [2], que traz o MPI pronto e dá boot nos workers pela rede. Porém, uma distribuição de cluster não faz o paralelismo nem divide os dados, tarefa do programa do grupo: ela apenas deixa pronta a configuração de que o MPI precisa (o mesmo MPI em todos os nós, rede, SSH sem senha entre eles e a lista de máquinas). Como essa configuração pode ser automatizada, o grupo optou pelo **Ubuntu Server**, uma distribuição de uso geral, como costuma ocorrer em clusters reais. Isso dá mais controle sobre cada peça e torna a montagem do cluster parte do aprendizado.

Ambiente por etapa. Docker e VMs são práticos, mas executam tudo sobre o mesmo hardware: com quatro VMs em um notebook de quatro núcleos, o ganho medido não representa um cluster real. Por isso, cada ambiente foi atribuído à etapa em que é mais útil:

Etapa	Ambiente	Objetivo
Desenvolvimento	Máquina local ou Docker	Escrever e depurar o código
Validação	VMs com Ubuntu Server, sem interface gráfica	Testar com rede e SSH reais e simular falhas desligando uma VM
Medição	Notebooks físicos do grupo em rede	Medir o *speedup* real, sem disputa de CPU

Tabela 2: Ambiente de cada etapa do projeto.

Automação. O **Vagrant** cria as VMs, e o **Ansible** configura os nós (MPI, chaves SSH, lista de máquinas e cópia do programa). O mesmo *playbook* do Ansible configura as VMs e os notebooks, bastando trocar a lista de máquinas, o que garante a mesma configuração na validação e na medição. O grupo já usou o Ansible em outras disciplinas.

Comunicação. A tecnologia escolhida é o **MPI** (*Message Passing Interface*), padrão de troca de mensagens em computação paralela, disponível em Python pela biblioteca `mpi4py`. Ele oferece as operações `scatter` e `gather`, que espalham os pedaços e recolhem os resultados, e mensagens ponto a ponto para a fila dinâmica. Sockets, gRPC e filas de mensagens (RabbitMQ) foram considerados, mas o MPI é o padrão da computação em cluster e já cobre o que o modelo precisa.

6 Métricas de sucesso e experimentos planejados

***Speedup*:** $S = T_1/T_N$, razão entre o tempo com 1 worker e com N workers. O ideal seria $S = N$, mas a parte que não se paraleliza (ler os dados, agregar, comunicar) limita o ganho (Lei de Amdahl).

Eficiência: $E = S/N$, quanto de cada worker adicional é aproveitado.

Distribuição da carga: pedaços processados e tempo ocioso de cada worker.

Corretude: o resultado distribuído deve ser idêntico ao de uma execução sequencial de referência, inclusive quando um worker falha.

Experimentos planejados:

1. **Escalabilidade:** tempo e *speedup* com 1, 2 e 4 workers, e o efeito do tamanho do pedaço, para identificar quando a comunicação passa a dominar.
2. **Estática × dinâmica** em um cluster heterogêneo (máquinas diferentes ou atraso artificial em um worker): tempo total, pedaços por worker e ociosidade.
3. **Tolerância a falhas:** derrubar um worker no meio da execução e verificar se o mestre reatribui o pedaço dele.

7 Cronograma

As entregas são semanais, às quintas-feiras, e a entrega final ocorre na última aula do semestre, em 03/12/2026.

Etapa	**Entrega**	**Data**
Pré-projeto	Apresentação da proposta	22/10/2026
Semana 1	Análise dos datasets; escolha do dataset e da pergunta	29/10/2026
Semana 2	Cluster de validação (VMs, Vagrant e Ansible)	05/11/2026
Semana 3	Versão sequencial de referência e primeira versão distribuída (estática)	12/11/2026
Semana 4	Fila dinâmica; experimento de escalabilidade	19/11/2026
Semana 5	Tolerância a falhas; experimento estático × dinâmico	26/11/2026
Final	Consolidação dos resultados e apresentação final	03/12/2026

Tabela 3: Cronograma das entregas.

Como o cronograma tem pouca folga, as semanas 1 e 2 correm em paralelo (datasets e cluster são independentes), e a última semana fica reservada para consolidar os resultados.

8 Divisão de papéis

Com três integrantes e seis semanas, o grupo optou por **papéis fixos**, um por frente de trabalho, em vez de papéis rotativos:

Papel	**Responsabilidades**
Dados e documentação	Análise e preparo dos datasets; consolidação dos relatórios; slides; registro das decisões
Desenvolvimento	Código MPI (mestre, workers, fila dinâmica, tolerância a falhas) e versão sequencial de referência
Infraestrutura e experimentos	Vagrant, Ansible e cluster; execução dos experimentos e coleta de métricas

Tabela 4: Papéis fixos do grupo.

A atribuição de cada papel ainda será definida pelo grupo. Mesmo com papéis fixos, a tendência é que todos participem um pouco de todas as frentes: o papel indica quem responde por cada uma, e não um limite para a colaboração. O trabalho é versionado em um repositório público no GitHub[1].

9 Riscos e desafios

Máquinas heterogêneas: na divisão estática, o worker mais lento determina o tempo total. *Mitigação:* fila dinâmica de tarefas.

Falha de workers: um nó pode cair no meio da execução. *Mitigação:* o mestre registra os pedaços de cada worker e reatribui os de quem parar de responder.

[1] `https://github.com/Matheus-Nardi/projeto-sd`

Custo de comunicação: pedaços pequenos demais fazem a rede superar o ganho do paralelismo. *Mitigação:* medir o efeito do tamanho do pedaço.

Configuração do ambiente: MPI, SSH e rede entre notebooks podem tomar mais tempo que o previsto. *Mitigação:* automação com Vagrant e Ansible e desenvolvimento inicial em uma máquina só.

Prazo curto: seis semanas com pouca folga. *Mitigação:* priorizar o *speedup* e a comparação estática × dinâmica, simplificando a tolerância a falhas se necessário.

Referências

[1] Calvin University. Dynamic load balancing (mpi4py). `https://pdcbook.calvin.edu/RaspberryPi-mpi4py/08DynamicLoadBalance/17dynamicLoad.html`. Acesso em: 21/10/2026.

[2] Michael Creel. PelicanHPC. `https://sourceforge.net/projects/pelicanhpc/`. Acesso em: 21/10/2026.

[3] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, 2004.

[4] INEP. Microdados do ENEM. `https://www.gov.br/inep/pt-br/acesso-a-informacao/dados-abertos/microdados/enem`. Acesso em: 21/10/2026.

[5] Java Design Patterns. Master-worker pattern. `https://java-design-patterns.com/patterns/master-worker-pattern/`. Acesso em: 21/10/2026.

[6] NYC Taxi and Limousine Commission. TLC trip record data. `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`. Acesso em: 21/10/2026.